Внимание

Необходимо **сбрасывать отчёт на почту** koliaulasevich@gmail.com

В теме письма указывается:
«ОС номер группы/подгруппы Фамилия И. О. № лабораторной работы»

Пример
«ОС 3/1 Уласевич Н. И. №4» или «ОС 3/1 Уласевич Н. И. №5»

На момент **сдачи 5ой лабораторной** необходимо **сбросить отчеты** по **4ой**

Внимание

Сдать лабораторную с заданием на **9-10 и 7-8** можно только в течении **4 пар** начиная с **26.10.26**

Лабораторная работа №5
Диспетчеризация и планирование

Цель работы: Получение практических навыков в планировании выполнения потоков в ОС.
!!!Задания являются накопительными, т.е. для выбранного уровня сложности нужно выполнить также все предыдущие.

Темы для предварительного изучения:

– Утилита Sysinternals Process Explorer (ссылка);
– Работа с приоритетами в Windows (лекция и документация) и POSIX (лекция и документация);
– Функция GetCurrentProcessorNumber (ссылка);
– Функция getcpu (ссылка);
– Файл scheduling.pdf (на diskstation);
– Multilevel Feedback Queue (ссылка).

Постановка задачи для Windows:
Уровень 4-5:

Приложение Lab-05a:

Простейшее консольное приложение, выводящее на консоль следующую информацию:

- Идентификатор текущего процесса;
- Идентификатор текущего потока;
- Класс приоритетов текущего процесса;
- Приоритет текущего потока;
- Приведенный в числовой вид приоритет потока;
- Маску родственности процесса в двоичном виде;
- Системную маску родственности в двоичном виде;
- Количество процессоров доступных для выполнения процессу;
- Номер процессора, назначенного для выполнения текущему потоку.

Приложение Lab-05x:

Простейшее консольное приложение, выполняющее цикл на 1 млн. итераций. Каждые 1000 итераций происходит задержка в 200мс, а также вывод следующей информации:

- Номер итерации;
- Идентификатор процесса;
- Идентификатор потока;
- Класс приоритета процесса;
- Приоритет потока;
- Приведенный в числовой вид приоритет потока
- Номер назначенного для выполнения процессора.

В конце своей работы приложение выводит на консоль время прошедшее с момента своего запуска (clock).

Приложение Lab-05b:

Простейшее консольное приложение, которое принимает следующие аргументы:

- P1 – целое число, задающее маску родственности процессоров;
- P2 – целое число, задающее класс приоритета первого дочернего процесса;
- P3 – целое число, задающее класс приоритета второго дочернего процесса.

Приложение должно выводить заданные параметры на консоль и запускать два одинаковых дочерних процесса на основе приложения Lab-05x, осуществляющих вывод в разные консольные окна и имеющие заданные приоритеты.

Требуется провести серию замеров со следующими параметрами:

P1	P2	P3
Все процессоры	Normal	Normal
Все процессоры	Below Normal	High
1 процессор	Below Normal	High

Во время работы приложения найти информацию о приоритетах процессов в Process Explorer.

Зафиксируйте (скриншот) в момент окончания одного из дочерних процессов расхождение в количестве выполненных процессами итераций. Для фиксации разницы допустимо значительно увеличить число итераций.

По зафиксированным скриншотам, объясните полученные результаты.

Приложение Lab-05c:

Простейшее консольное приложение, которое принимает следующие аргументы:

- P1 – целое число, задающее маску родственности процессоров;
- P2 – целое число, задающее класс приоритета процесса;
- P3 – целое число, задающее класс приоритета первого дочернего потока;
- P4 – целое число, задающее класс приоритета второго дочернего потока.

Приложение содержит потоковую функцию аналогичную приложению Lab-05x.

Приложение должно выводить заданные параметры на консоль и запускать два одинаковых дочерних потока, осуществляющих вывод в консольное окна и имеющие заданные приоритеты.

Требуется провести серию замеров со следующими параметрами:

P1	P2	P3	P4
Все процессоры	Normal	Normal	Normal

Все процессоры	Normal	Lowest	Highest
1 процессор	Normal	Lowest	Highest

Во время работы приложения найти информацию о приоритетах потоках в Process Explorer.

Зафиксируйте (скриншот) в момент окончания одного из дочерних потоков расхождение в количестве выполненных потоками итераций. Для фиксации разницы допустимо значительно увеличить число итераций.

По зафиксированным скриншотам, объясните полученные результаты.

Постановка задачи для Linux:

Уровень 4-5:

Приложение Lab_05a:

Приложение идентично приложению из заданий по Windows за исключением пункта о системной маске родственности (в Linux её выводить не требуется).

Приложение Lab-05x:

Простейшее консольное приложение, выполняющее цикл на 1 млн. итераций. Каждые 1000 итераций происходит задержка в 200мс, а также вывод следующей информации:

- Номер итерации;
- Идентификатор процесса;
- Идентификатор потока;
- Уровень любезности;
- Номер назначенного для выполнения процессора.

В конце своей работы приложение выводит на консоль время прошедшее с момента своего запуска (clock).

Приложение Lab-05b:

Простейшее консольное приложение, которое принимает следующие аргументы:

- P1 – целое число, задающее маску родственности процессоров;
- P2 – целое число, задающее класс приоритета первого дочернего процесса;
- P3 – целое число, задающее класс приоритета второго дочернего процесса.

Приложение должно выводить заданные параметры на консоль и запускать два одинаковых дочерних процесса на основе приложения Lab-05x, осуществляющих вывод в консольное окно и имеющие заданные приоритеты (через nice).

Требуется провести серию замеров со следующими параметрами:

P1	P2	P3
Все процессоры	0	0
Все процессоры	19	-20
1 процессор	19	-20

Во время работы приложения найти информацию о приоритетах процессов через утилиту ps и каталог /proc. Через каталог /proc также найти информацию о маске родственности.

Зафиксируйте (скриншот) в момент первого окончания одного из дочерних процессов расхождение в количестве выполненных процессами итераций.

По зафиксированным скриншотам, объясните полученные результаты.

Приложение Lab-05c:

Простейшее консольное приложение, которое принимает следующие аргументы:

- P1 – целое число, задающее маску родственности процессоров;
- P2 – целое число, задающее класс приоритета первого дочернего потока;
- P3 – целое число, задающее класс приоритета второго дочернего потока.

Приложение содержит потоковую функцию аналогичную приложению Lab-05x.

Приложение должно выводить заданные параметры на консоль и запускать два одинаковых дочерних потока, осуществляющих вывод в консольное окна и имеющие заданные приоритеты (через setpriority).

Требуется провести серию замеров со следующими параметрами:

P1	P2	P3
Все процессоры	0	0
Все процессоры	19	-20
1 процессор	19	-20

Зафиксируйте (скриншот) в момент окончания одного из дочерних потоков расхождение в количестве выполненных потоками итераций.

По зафиксированным скриншотам, объясните полученные результаты.

Сравнить результаты между процессами и потоками и сделать выводы.

Постановка задачи без привязки к конкретной ОС:

Уровень 7-8:

Приложение Lab-05SJF:

Приложение реализует алгоритм планирования Shortest Job First.

Приложение Lab-05RR:

Приложение реализует алгоритм планирования Round Robin.

Приложение Lab-05CP:

Приложение реализует алгоритм вытесняющего приоритетного планирования.

Все приложения разрабатываются в простом виде, то есть по факту никакие планируемые задачи выполняться не будут. По сути, ваши приложения являются планировщиком, который получает на вход некоторые необходимые параметры и выдаёт план выполнения задач.

Для примера, провести проверку построения планов выполнения алгоритмов для следующих задач.

Номер задачи	Время поступления	Время выполнения	Приоритет
1	0	80	3
2	20	45	5
3	30	60	2
4	50	30	4
5	70	90	1
6	100	55	3
7	120	75	2
8	150	40	5
9	180	65	4
10	200	85	1

Для алгоритмов Round Robin и на основе приоритетов провести расчёты при разных значениях кванта и определить оптимальный на ваш взгляд. Пояснить свой выбор!

Для каждого из алгоритмов высчитывается статистика: среднее время ожидания, среднее время выполнения, среднее время оборота, пропускная способность.

Сравнить результаты по всем трём алгоритмам и сделать выводы. Для сравнения из алгоритмов Round Robin и на основе приоритетов берутся значения при оптимальных на ваш взгляд квантах.

Уровень 9-10:

Приложение Lab-05MQ:

Приложение эмулирует работу планировщика с алгоритмом Multilevel Feedback Queue.

Необходимо создать эмулятор планировщика, который будет:

- Разделять процессы по разным очередям в зависимости от их потребностей в CPU;
- Перемещать процессы между очередями;
- Отдавать приоритет процессам, которые быстро используют CPU;
- Учитывать процессы с большим количеством операций ввода-вывода.

Предусмотреть как минимум три очереди:

➢ Высокоприоритетная очередь (Q0 – потребляет 50% процессорного времени):

- Предназначена для интерактивных задач;
- Короткий квант времени (10-20 мс);
- Алгоритм планирования: Round Robin;
- Задачи попадают сюда при создании;
- Если задача не использует весь квант – остается здесь.

➢ Средняя приоритетная очередь (Q1 - потребляет 30% процессорного времени):

- Для задач со средней интенсивностью CPU;
- Увеличенный квант времени (40-80 мс);
- Алгоритм: Round Robin;
- Задача понижается сюда, если исчерпала квант в Q0;
- Может повышаться обратно при I/O операциях.

➢ Низкоприоритетная очередь (Q2 - потребляет 20% процессорного времени):

- Для CPU-интенсивных задач;
- Большой квант времени (100-200 мс);
- Алгоритм: FCFS;
- Задача попадает сюда, если исчерпала кванты в предыдущих очередях;
- С равными шансами процесс может переместиться в одну из предыдущих очередей или остаться в этой, если исчерпал квант в этой очереди.

Исходные данные:

➢ Количество процессов: 100;
➢ Время выполнения процессов: случайное;
➢ Типы процессов: CPU-bound и I/O-bound.

Обязательные компоненты программы

Структура процесса: ID процесса, время поступления, время выполнения, текущая очередь, время ожидания и т.д. (можно добавлять всё что требуется для работы вашей реализации)

Статистика для сбора: среднее время ожидания, общее время выполнения, количество переключений, процент использования CPU, количество перемещений между очередями.

После успешного планирования всех процессов (т.е. когда они все завершат своё выполнение), для каждой реализованной очереди планирования программа должна выводить следующую информацию по всем процессам:

➢ Идентификатор процесса (PID);
➢ Время поступления процесса в систему;
➢ Время выполнения;
➢ Время ожидания;
➢ Оборотное время;

- Статистические показатели.

Для каждого очереди необходимо рассчитать и вывести:

- Среднее время ожидания (Average Waiting Time);
- Среднее оборотное время (Average Turnaround Time).

Вопросы для самоконтроля:

1. Поясните понятия планирование и диспетчеризация?
2. Опишите процесс диспетчеризации?
3. Что такое планировщик? Какие бывают планировщики? Опишите каждый из них.
4. Какой из планировщиков определяет степень мультизадачности ОС?
5. Какие виды алгоритмов планирования вы знаете?
6. Перечислите общие и специфичные задачи планирования.
7. Поясните понятие «циклическое планирование».
8. Поясните понятие «приоритетное планирование».
9. Поясните понятие «приоритет процесса».
10. Что такое базовый приоритет потока, как он вычисляется и диапазон его изменения?
11. Поясните принцип использования значения nice – процесса, диапазон его изменения.
12. Поясните понятие «кооперативное планирование».
13. Поясните понятие «родственное планирование».
14. Что такое маска родственности? Как задать?
15. Что такое перепланировка? Когда наступает момент перепланировки?